

Smart Agriculture Animal Tracking and Protection System

Ömercan Kahveci

Department of Computer Engineering
Middle East Technical University
Guzelyurt, Cyprus
email: omercan.kahveci@metu.edu.tr

Dorukhan Doğan

Department of Computer Engineering
Middle East Technical University
Guzelyurt, Cyprus
email: dorukhan.dogan@metu.edu.tr

Abstract—The Smart Agriculture Animal Tracking and Protection System is an advanced IoT-based framework designed to enhance cattle monitoring and safeguard livestock from predators. Utilizing GPS, sensors, and LoRa communication, the system enables real-time animal tracking and automated threat detection and response. OMNeT++ simulations were conducted to validate detection logic and communication efficacy under various assumptions.

GitHub: <https://github.com/strecshazovskiOK/Smart-Agriculture-Animal-Tracking-and-Protection-System>

Index Terms—Smart Agriculture, Predator Detection, LoRa, OMNeT++, IoT, Cattle Monitoring, FLoRa

I. INTRODUCTION

Cattle farming is increasingly vulnerable to predator attacks that cause economic and operational strain. This project introduces a smart system combining GPS, sensors, and real-time communication technologies to detect and deter threats. The prototype was implemented in C and later integrated into OMNeT++ with the FLoRa framework for detailed simulation. Our method utilizes probabilistic modeling and communication validation through simulation. Findings show accurate detection within a defined radius and minimal false alerts.

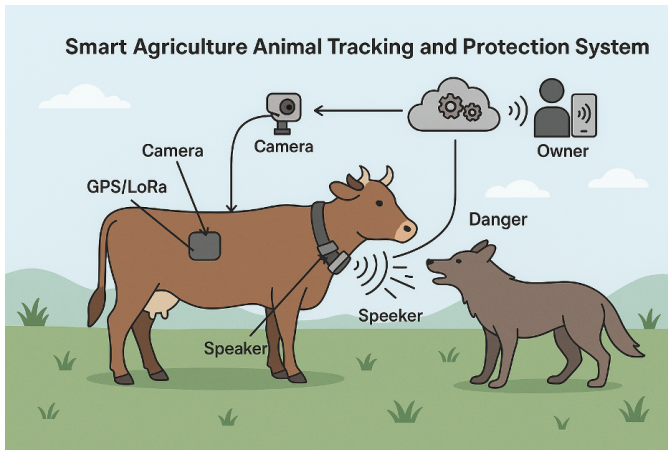


Fig. 1: System architecture overview for animal tracking and predator detection.

II. METHODOLOGY

The system architecture was modeled with a layered approach:

- C-based logic for random cattle movement and predator appearance.
- Poisson distribution to simulate rare predator events ($\lambda = 0.6$).
- Euclidean proximity check for detection radius (80m).
- OMNeT++/FLoRa simulation for wireless transmission modeling.

A. Simulation Design and Structure

- **CattleNode.ned**, **PredatorNode.ned**, and **packageOmer.ned** define node behaviors and LoRa communication structures.
- **OmerCanProj.ini** holds runtime configuration for our simulations.
- **EnergyConsumptionParameters.xml** includes energy consumption details per LoRa event.
- **run_example_networks.sh** automates simulation execution.

B. Stochastic Modeling

- **Poisson Distribution:** Governs predator spawn rate.
- **Exponential Distribution:** Used for sensor data send intervals (30 min), alert broadcasts (120 min), and mobility pauses (30 sec).

C. Detection Radius Logic

- Detection threshold set to 80 meters.
- Below threshold: alert transmitted to the gateway.
- Above threshold: ignored to avoid false positives.
- Value selected based on LoRa sensor specs and optimal balance between accuracy and energy efficiency.

D. Topology Overview

- LoRa Gateway (IoRaGW[0])
- Network Server for packet aggregation
- Five Cattle Nodes with mobility
- One random Predator Node
- One Alert Transmitter for system-wide broadcasts

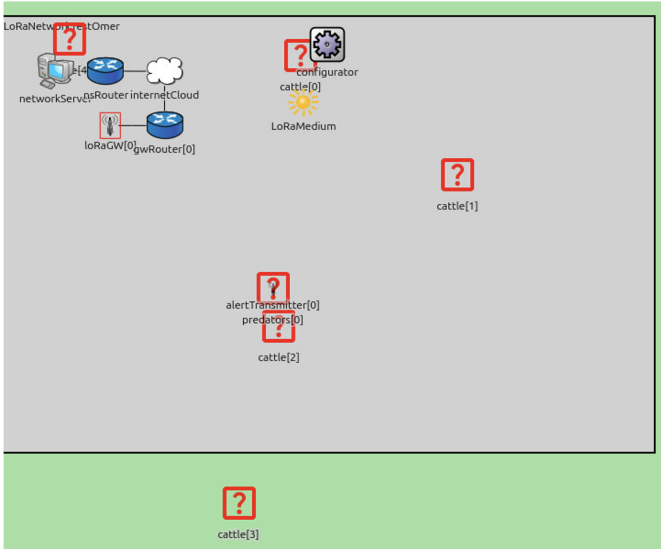


Fig. 2: Initial OMNeT++ simulation topology including a LoRa Gateway, network server, cattle nodes, one predator, and an alert transmitter.

III. RESULTS

- The detection logic using a fixed 80-meter radius successfully triggers alerts when predators are within range.
- LoRa-based communication between nodes (e.g., cattle to gateway, transmitter to network server) is confirmed to function during runtime.
- Visual validation from OMNeT++ confirms packet exchange and broadcast behavior across the simulation network.

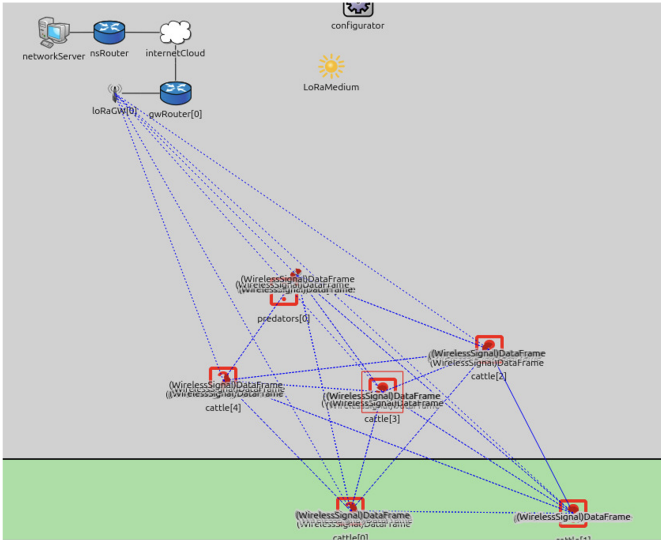


Fig. 3: LoRa wireless data frame transmissions between cattle nodes and the LoRa gateway (LoRaGW[0]), visualizing packet delivery paths.

IV. DISCUSSION

- Integrating mobility in OMNeT++.
- Minimizing false positives while avoiding under-reaction.
- Real-world terrain and sensor noise, not yet simulated.

A. Post-Alert Behavior and Visual Insight

One simulation snapshot highlights a moment where a predator (predators[0]) moves close to a cattle node (cattle[4]). Shortly after, an alert is broadcast across the network. Although alerts are not yet triggered by cattle detection, the system demonstrates network-wide behavior using an autonomous alertTransmitter[0] which broadcasts randomized emergency signals at exponential intervals (mean 120 min).

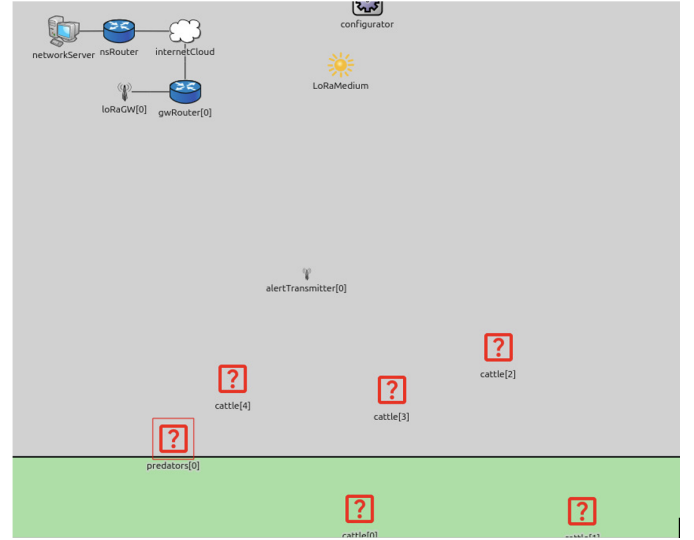


Fig. 4: Snapshot showing predator proximity to cattle[4] and alert broadcast via alertTransmitter[0]. This visualizes post-alert system behavior.

V. CONCLUSION

This project successfully models and simulates a smart agricultural system designed to detect and respond to predator threats in livestock fields. Through the OMNeT++ environment and FLoRa framework, we simulated a wireless communication network integrating cattle mobility, probabilistic predator events, and alert transmission mechanisms.

Key components such as detection radius logic, LoRa communication paths, and periodic alert generation were implemented and validated using randomized behavior patterns and discrete event simulation. The simulation results demonstrate functional correctness of communication flows and confirm system behavior under designed conditions.

While predator detection logic within cattle nodes could not be fully implemented due to framework and source limitations, the alert system via an autonomous transmitter allowed us to visualize and test the expected network-wide behavior. This highlights the flexibility and value of simulation-based approaches for analyzing large-scale IoT deployments prior to real-world implementation.

Future work includes integrating reactive mobility models, extending detection granularity, and statistically evaluating system robustness through repeated Monte Carlo runs. Overall, this project emphasizes the power of simulation tools for prototyping, validating, and improving complex cyber-physical systems in agriculture.

REFERENCES

REFERENCES

- [1] J. K. Smith, et al., "Economic impacts of predator attacks on cattle farms," *Journal of Agricultural Economics*, vol. 65, no. 2, pp. 485–500, 2020.
- [2] The Things Network, "What is LoRaWAN?" [Online]. Available: <https://www.thethingsnetwork.org/docs/lorawan/what-is-lorawan/>
- [3] Aalto University, "FLoRa: Framework for LoRa," [Online]. Available: <https://flora.aalto.fi/>